

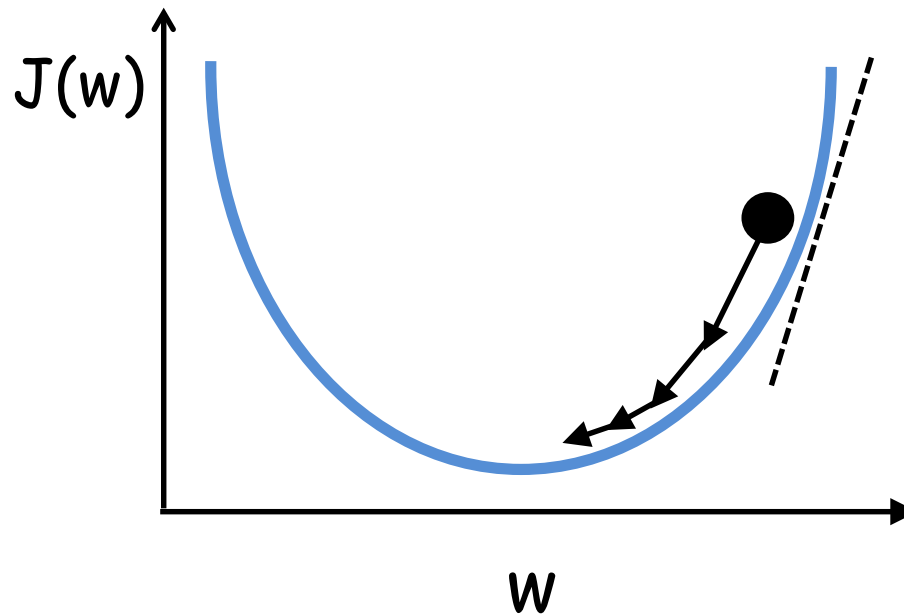
Neural Network

6. 학습 기술들

6가지 알고리즘 문제 풀어보기.

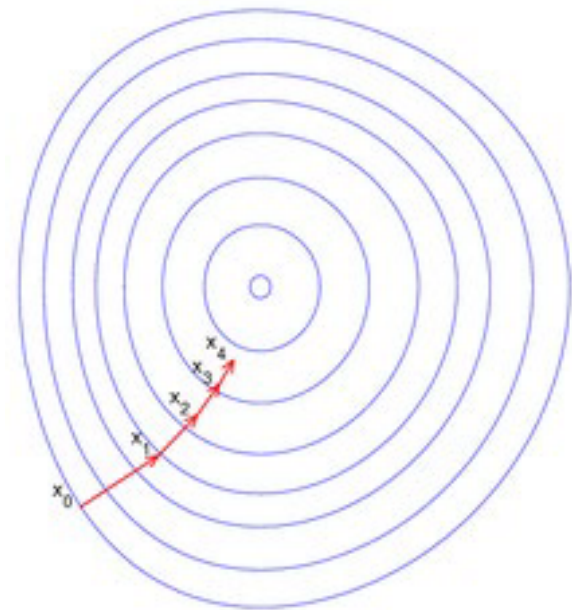
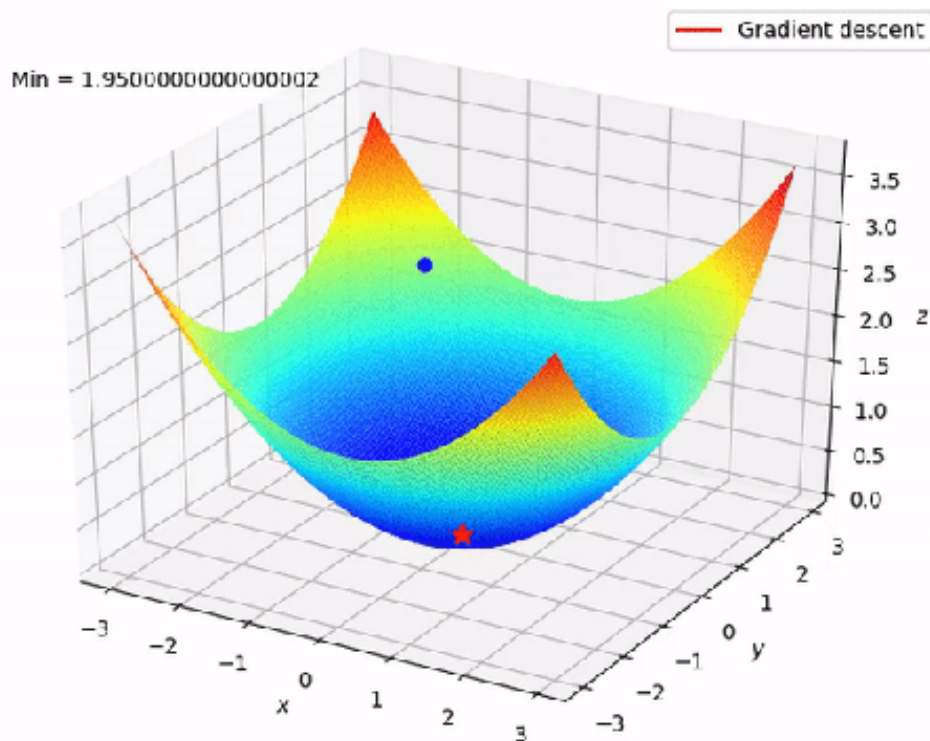
Loss function

- 인공신경망 학습 = Loss function 을 낮추는 과정



Gradient Descent Method (1)

$$\mathbf{x}_{n+1} = \mathbf{x}_n - \eta \cdot \nabla f(\mathbf{x}_n)$$



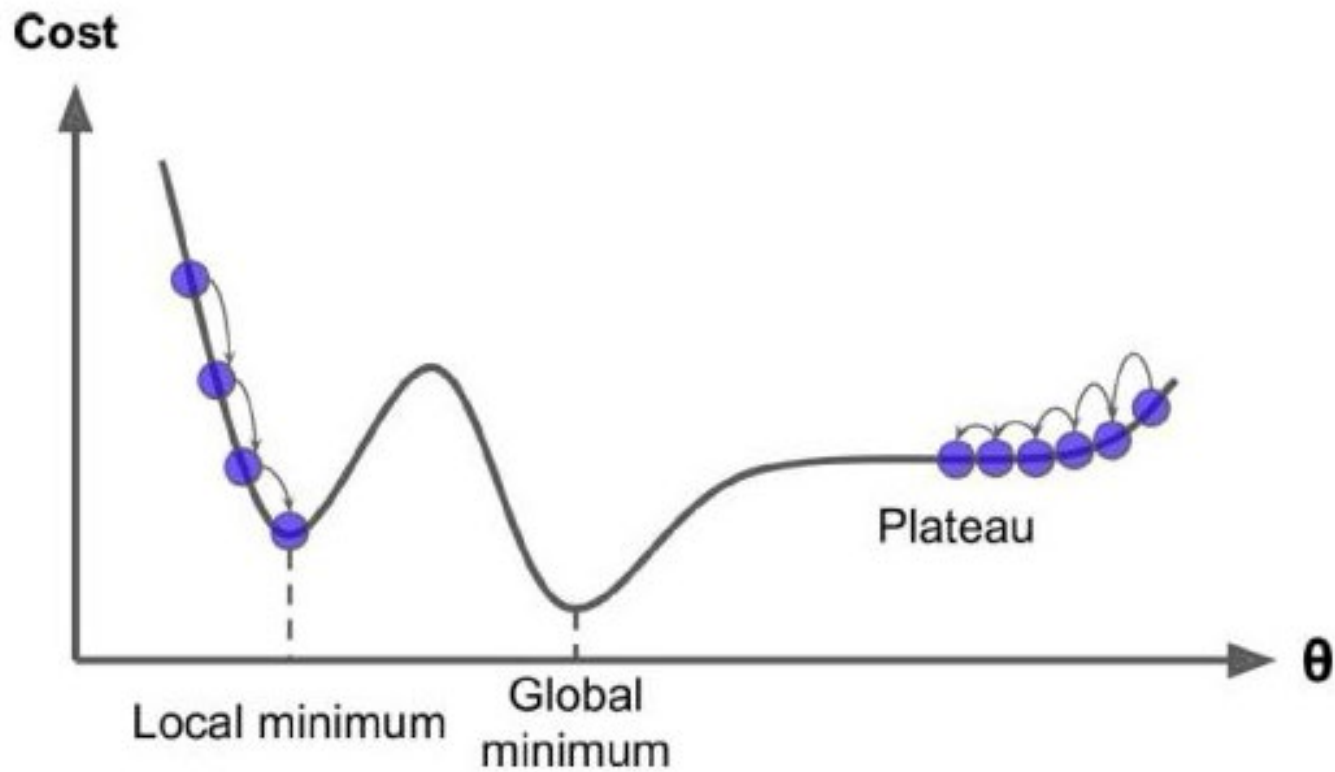
Example

$$f(x, y) = x^2 + 4y^2$$

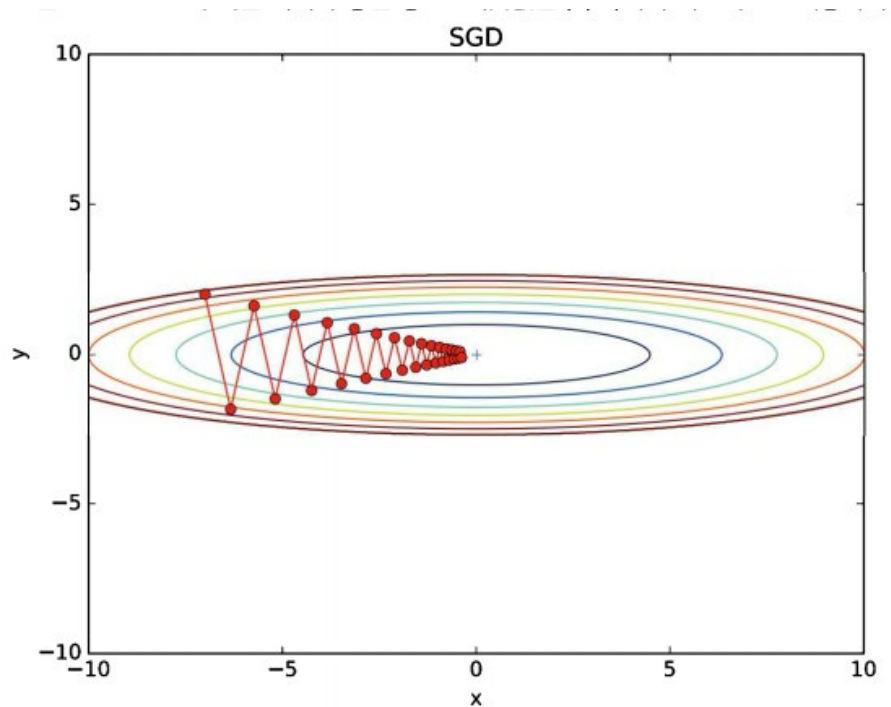
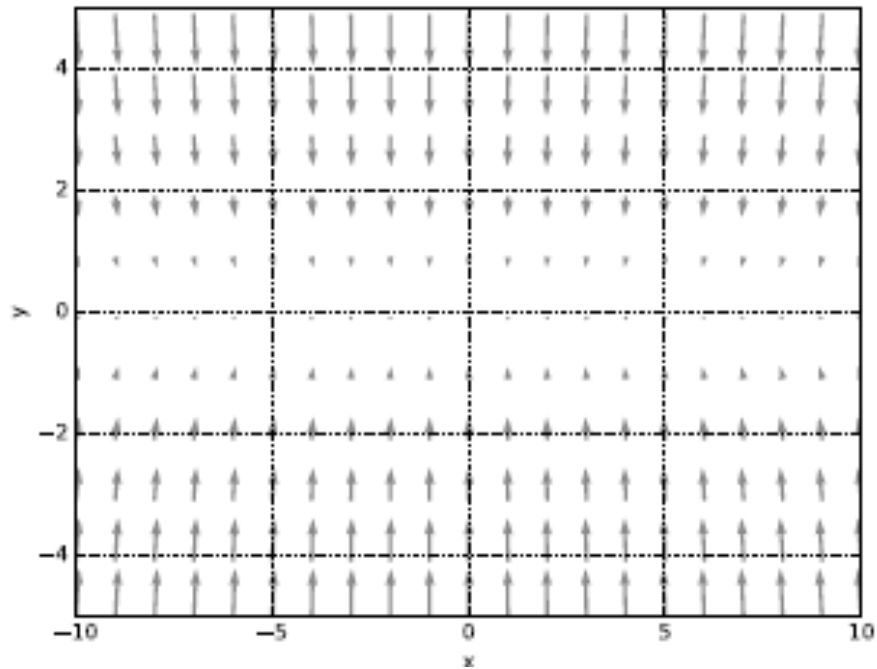
$$\mathbf{x}_0 = (4, 2), \quad \eta = \frac{1}{4}$$

$$\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3 = ?$$

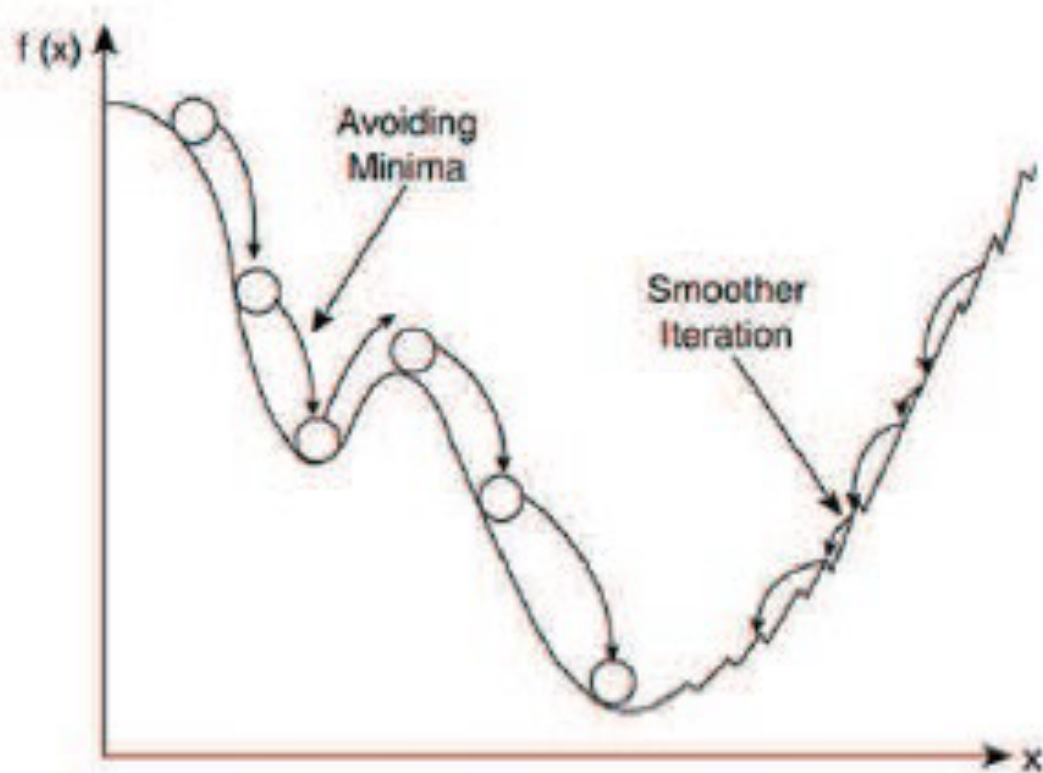
- Local minimum 또는 saddle point 도출
- 초기 위치 중요



- Gradient 방향은 Level curve 와 수직
- 최적점 방향과 Gradient 방향의 차이가 클 수 있음



Momentum



Momentum (N.A)

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{v}_n$$

$$\mathbf{v}_n = -\eta \cdot \nabla f(\mathbf{x}_n)$$

) Descent



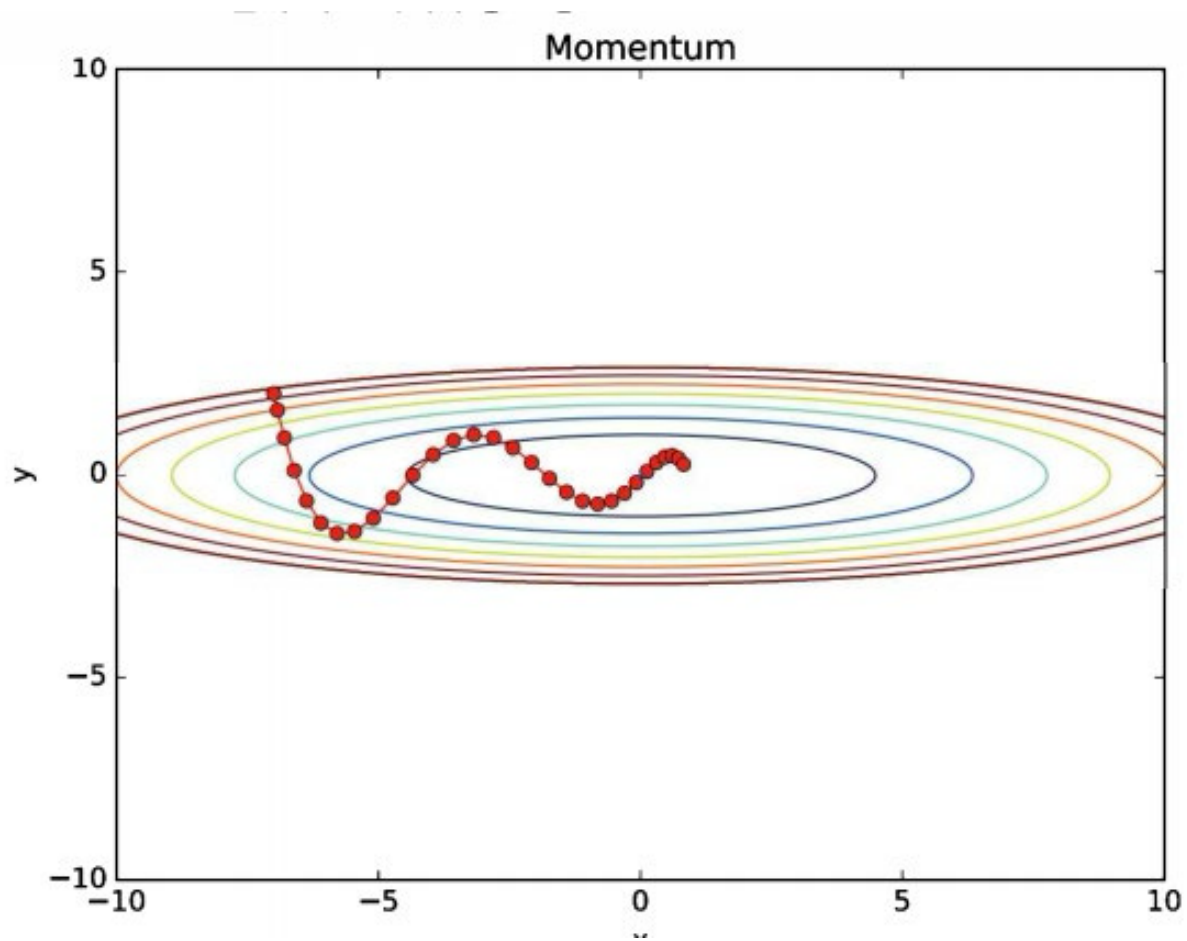
$$\mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{v}_n$$

$$\mathbf{v}_n = \alpha \mathbf{v}_{n-1} - \eta \cdot \nabla f(\mathbf{x}_n)$$

$$\mathbf{v}_{-1} = \mathbf{0}$$

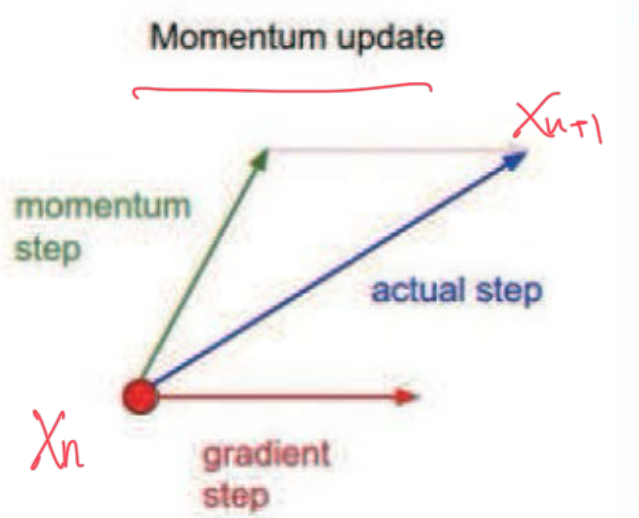
α 계수를 유지한다.
($0 < \alpha < 1$)

Momentum

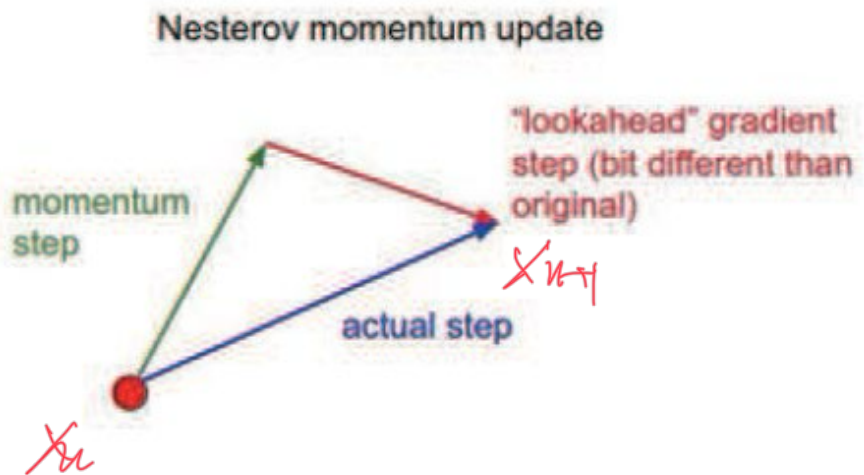


NAG

- Nesterov Accelerated Gradient
- Gradient를 미래 지향적으로 반영



⇒ Descent + momentum
현재 위치점



⇒ 미래 위치점 방향을 살짝

NAG (β, α)

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{v}_n$$

$$\mathbf{v}_n = \alpha \mathbf{v}_{n-1} - \eta \cdot \nabla f(\mathbf{x}_n)$$

$$\mathbf{v}_{-1} = \mathbf{0}$$

momentum

Descent

$\mathbf{x}_n$: 현재 위치

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{v}_n$$

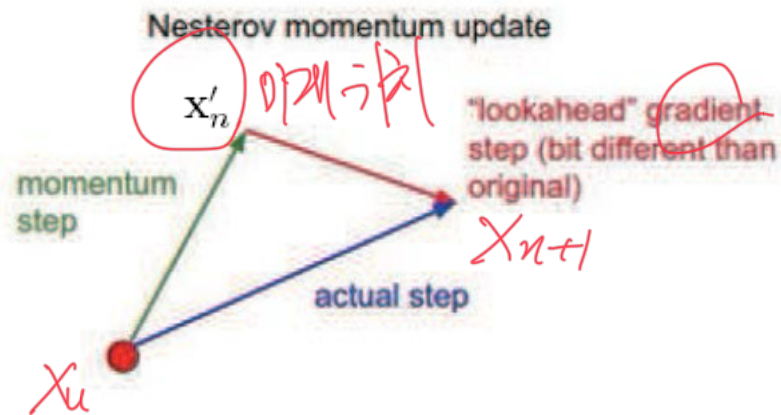
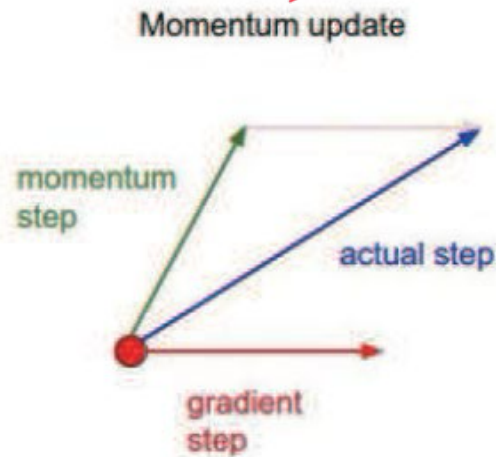
$$\mathbf{v}_n = \alpha \mathbf{v}_{n-1} - \eta \cdot \nabla f(\mathbf{x}_n + \alpha \mathbf{v}_{n-1})$$

$$\mathbf{v}_{-1} = \mathbf{0}$$

momentum

현재 위치 + 다음 위치

Bengio's approximation



$$\mathbf{x}'_{n+1} = \mathbf{x}'_n - \eta \nabla f(\mathbf{x}'_n) + \alpha \mathbf{v}_n$$

$$= \mathbf{x}'_n - \eta \nabla f(\mathbf{x}'_n) + \alpha (\alpha \mathbf{v}_{n-1} - \eta \nabla f(\mathbf{x}'_n))$$

$$= \mathbf{x}'_n - (1 + \alpha) \eta \nabla f(\mathbf{x}'_n) + \alpha^2 \mathbf{v}_{n-1}$$

$$\mathbf{v}_n = \alpha \mathbf{v}_{n-1} - \eta \nabla f(\mathbf{x}'_n), \quad \mathbf{v}_{-1} = \mathbf{0}, \quad \mathbf{x}'_0 = \mathbf{x}_0$$

AdaGrad (16)

- learning rate 를 axis 별로, step 별로 변화
- step 이 지날수록 learning rate 를 줄임
- 큰 변화 axis 는 작은 learning rate
- 작은 변화 axis 는 큰 learning rate 로 설정

$$\mathbf{x}_{n+1} = \mathbf{x}_n - \eta \frac{1}{\sqrt{\mathbf{h}_n}} \odot \nabla f(\mathbf{x}_n)$$

$$\mathbf{h}_n = \mathbf{h}_{n-1} + \nabla f(\mathbf{x}_n) \odot \nabla f(\mathbf{x}_n), \quad \mathbf{h}_{-1} = \mathbf{0}$$

$h_n \leftarrow (x_n)^2 \Rightarrow$ 계승 추가 누적합

AdaGrad

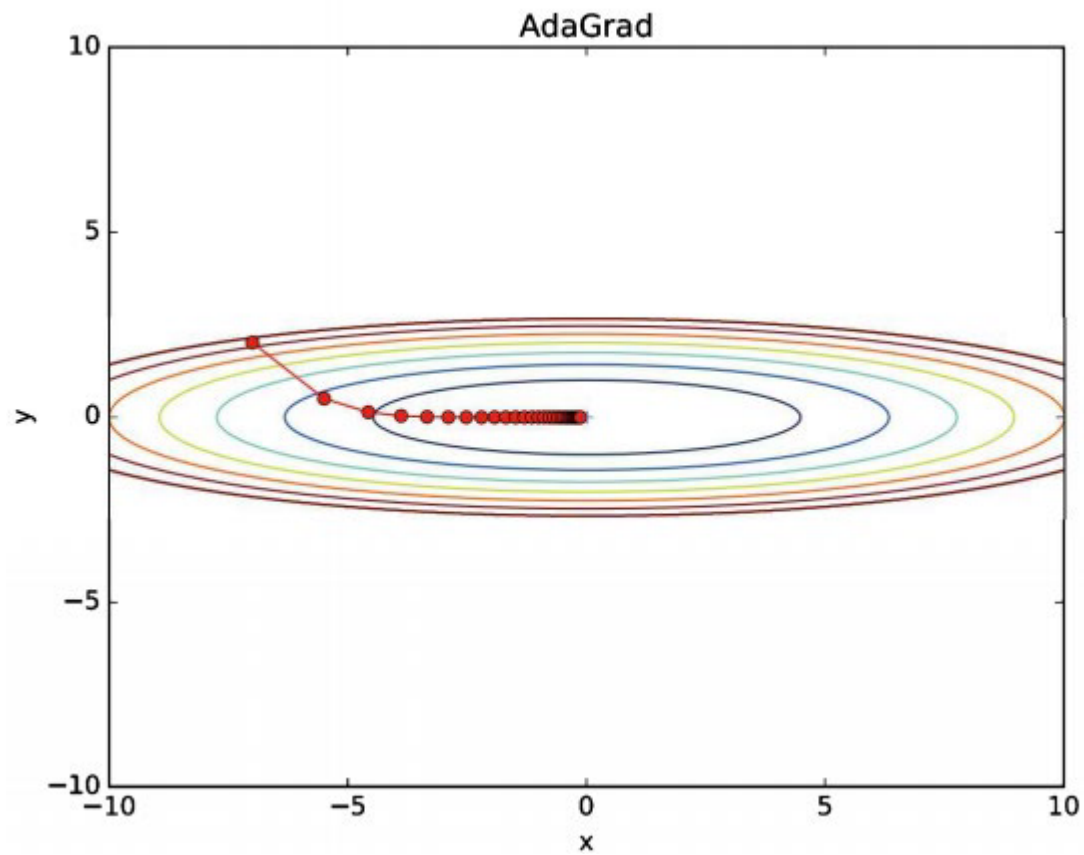
- Hadamard product

$$(a, b) \odot (\alpha, \beta) = (a\alpha, b\beta), \quad \begin{pmatrix} a & b \\ c & d \end{pmatrix} \odot \begin{pmatrix} \alpha & \beta \\ \gamma & \delta \end{pmatrix} = \begin{pmatrix} a\alpha & b\beta \\ c\gamma & d\delta \end{pmatrix}$$

$$\frac{1}{\sqrt{(a, b)}} = \left(\frac{1}{\sqrt{a}}, \frac{1}{\sqrt{b}} \right), \quad \frac{1}{\sqrt{\begin{pmatrix} a & b \\ c & d \end{pmatrix}}} = \begin{pmatrix} \frac{1}{\sqrt{a}} & \frac{1}{\sqrt{b}} \\ \frac{1}{\sqrt{c}} & \frac{1}{\sqrt{d}} \end{pmatrix}$$

$$\mathbf{h}_n = \sum_{k=0}^n \nabla f(\mathbf{x}_k) \odot \nabla f(\mathbf{x}_k)$$

AdaGrad



RMSProp (γ)

- AdaGrad의 업데이트 $\rightarrow$ 제곱 누적이 커지면서 방지
- 학습률을 구할 때 누적치와 현재 gradient의 가중 평균
- 최근 gradient 값을 더 반영

$$\mathbf{x}_{n+1} = \mathbf{x}_n - \eta \frac{1}{\sqrt{\mathbf{h}_n}} \odot \nabla f(\mathbf{x}_n)$$

$$\mathbf{h}_n = \underbrace{\gamma}_{\text{forgetting factor}} \mathbf{h}_{n-1} + (1 - \gamma) \nabla f(\mathbf{x}_n) \odot \nabla f(\mathbf{x}_n), \quad \mathbf{h}_{-1} = \mathbf{0}$$

forgetting factor

- γ : forgetting factor, decay rate

Adam (β_1, β_2)

- Momentum + RMSProp
- 가장 널리 쓰이는 optimizer
- 가중치 β_1, β_2 로 점화식 변형

$$\mathbf{x}_{n+1} = \mathbf{x}_n - \eta \frac{1}{\sqrt{\hat{\mathbf{h}}_n}} \odot \hat{\mathbf{m}}_n$$

RMS (pointing to $\sqrt{\hat{\mathbf{h}}_n}$)
Momentum (pointing to $\hat{\mathbf{m}}_n$)

$$\mathbf{m}_n = \beta_1 \mathbf{m}_{n-1} + (1 - \beta_1) \nabla f(\mathbf{x}_n), \quad \mathbf{m}_{-1} = \mathbf{0}$$

Momentum (pointing to $\beta_1 \mathbf{m}_{n-1}$)
? (pointing to $(1 - \beta_1)$)

$$\mathbf{h}_n = \beta_2 \mathbf{h}_{n-1} + (1 - \beta_2) \nabla f(\mathbf{x}_n) \odot \nabla f(\mathbf{x}_n), \quad \mathbf{h}_{-1} = \mathbf{0}$$

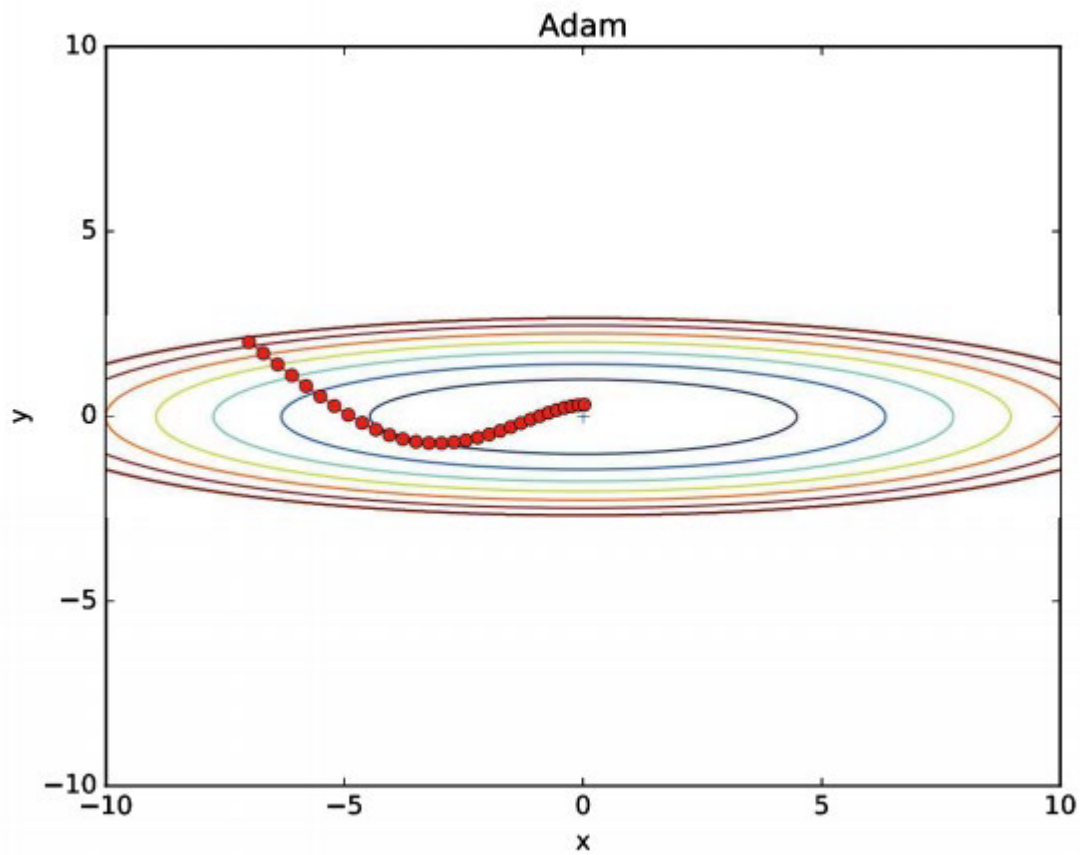
변분 (pointing to $\beta_2 \mathbf{h}_{n-1}$)
제곱 (pointing to $\nabla f(\mathbf{x}_n) \odot \nabla f(\mathbf{x}_n)$)

$$\hat{\mathbf{m}}_n = \frac{\mathbf{m}_n}{1 - \beta_1^{n+1}}$$

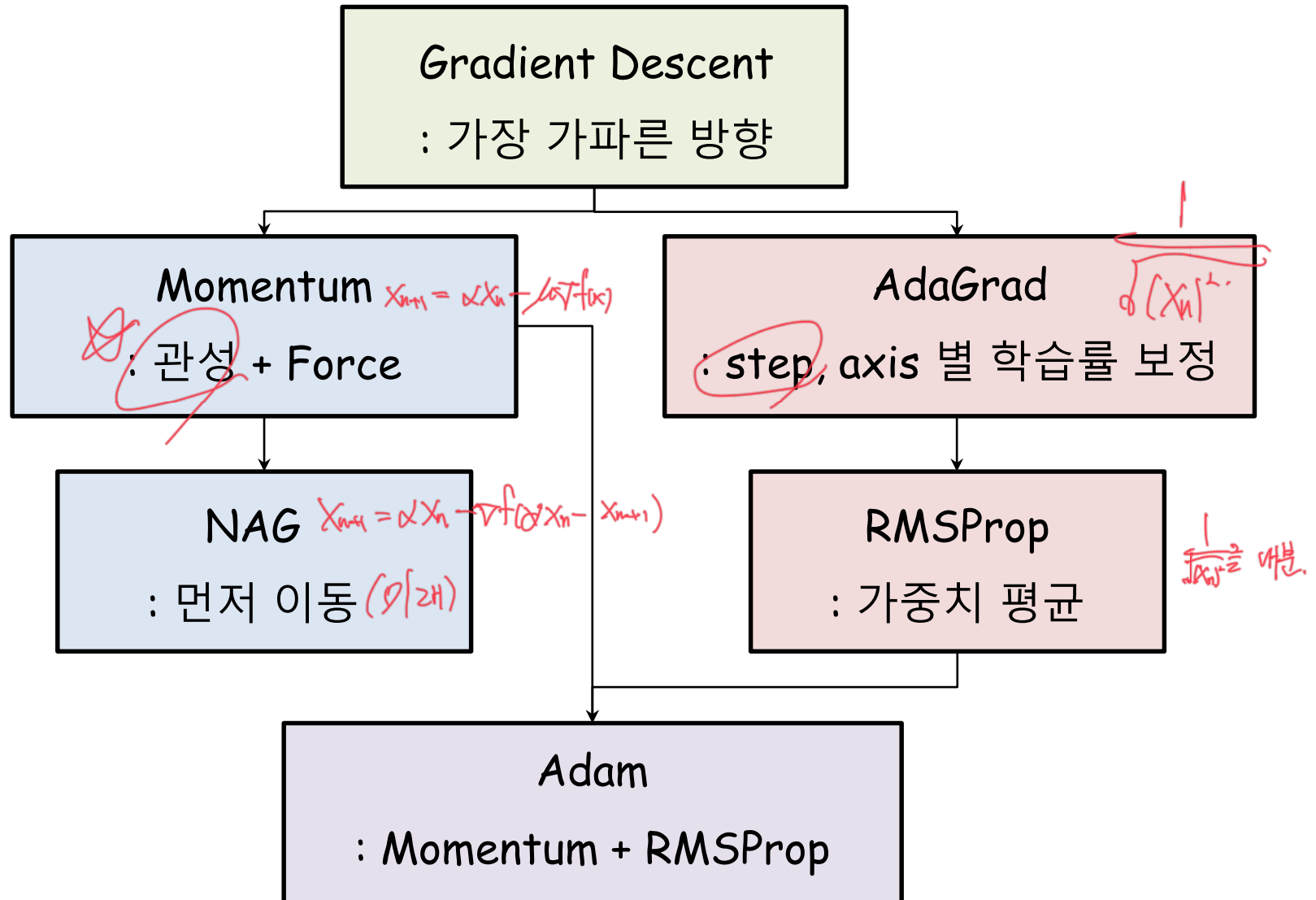
$$\hat{\mathbf{h}}_n = \frac{\mathbf{h}_n}{1 - \beta_2^{n+1}}$$

step 당 사용
보정값 사용

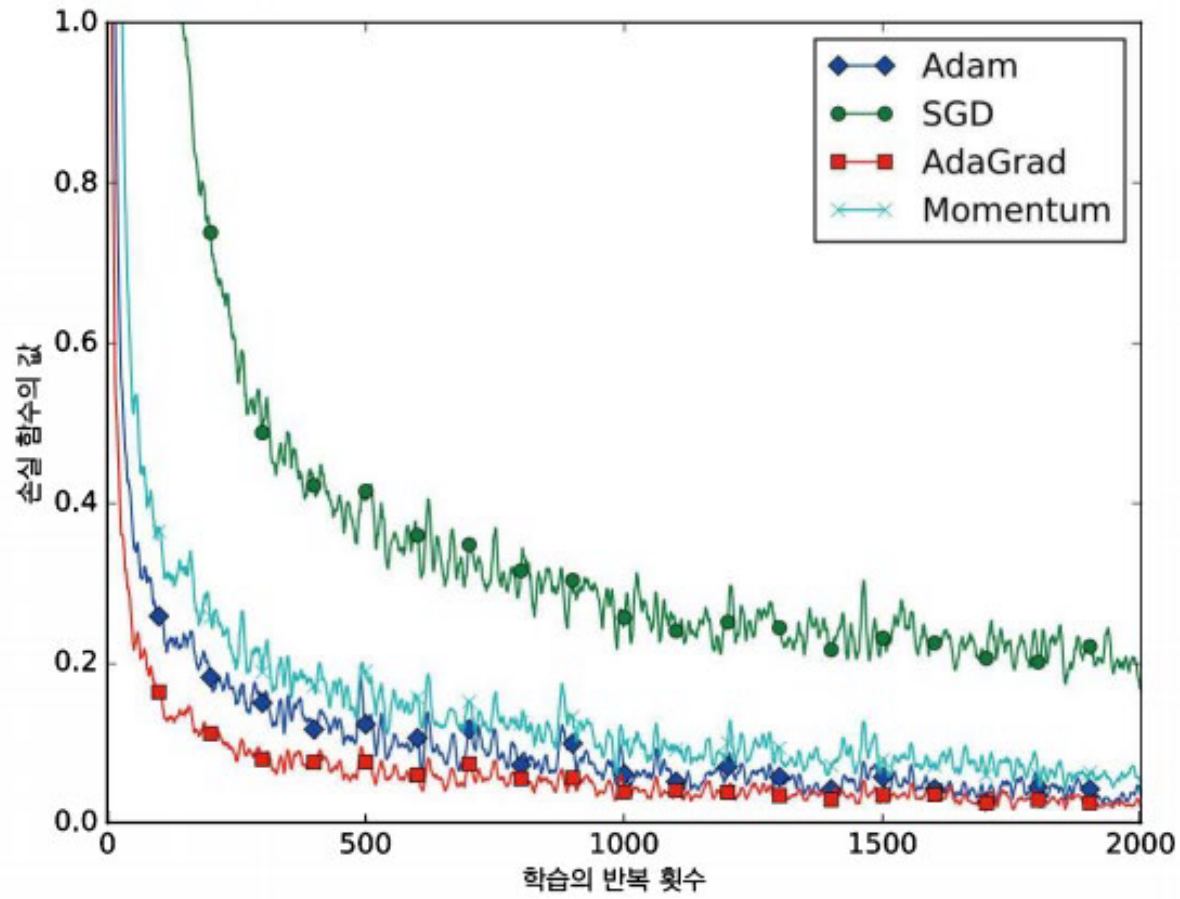
Adam



Optimizer (실습코드로 확인)

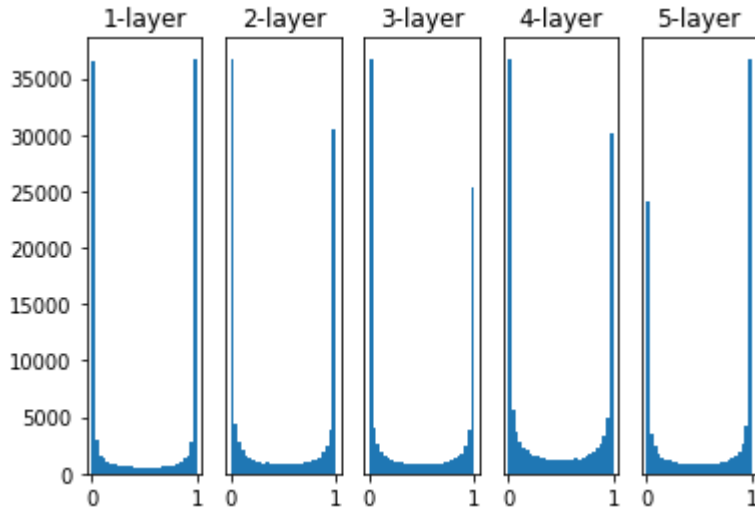


Optimizer



Initialization (초기화)

W가 클 경우 : $W_{init} \sim N(0, 1^2)$

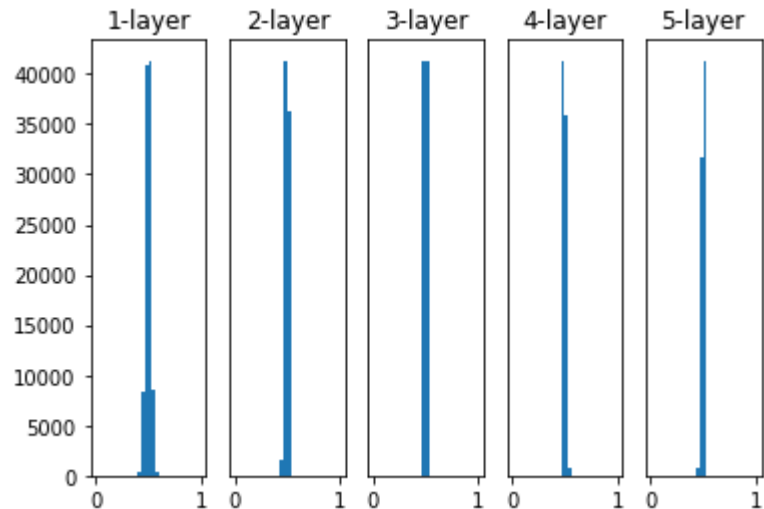


Vanishing Gradient Problem

W가 너무 커도

○ 사디언 1로 가옴

W가 작을 경우 : $W_{init} \sim N(0, 0.01^2)$

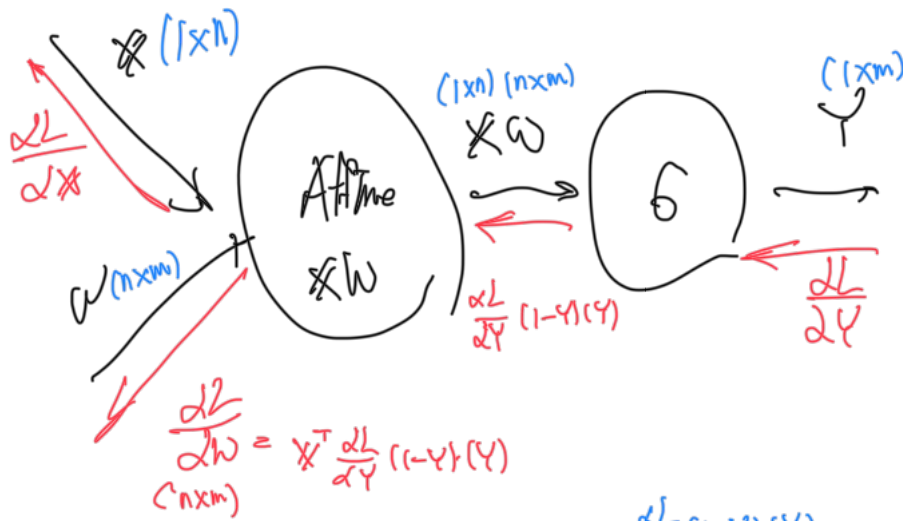


표현력 제한

초기화도 너무 0으로 가깝다면 :

$$\sum_{i=1}^n \frac{1}{1 + e^{-x_i}} = 0 \quad \therefore \sigma(x) = \frac{1}{2}$$

sigmoid



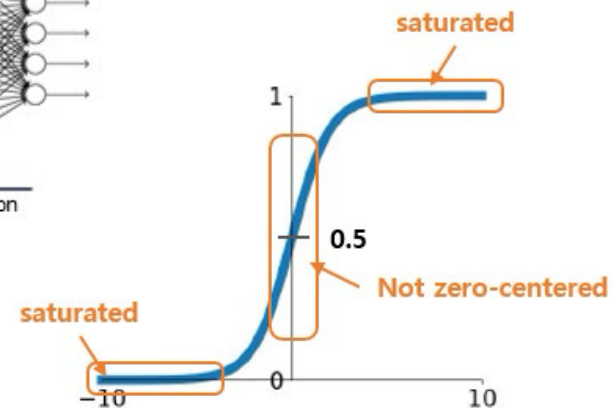
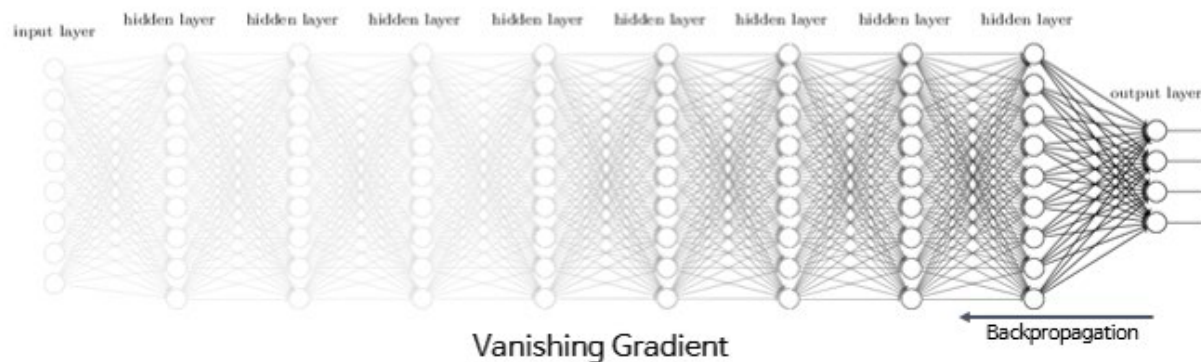
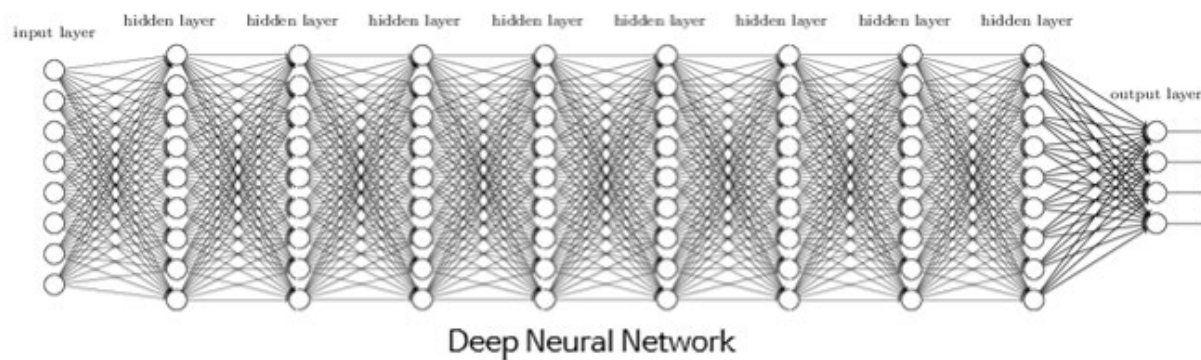
① $y=0.1$ 일때. $\Rightarrow$ 미분값이 매우 작아진다 $\therefore \frac{\partial L}{\partial y} (1-y)(y)$

② $y=\frac{1}{2}$ 일때. $\Rightarrow$ 표현력 제한.

$$x^T \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{pmatrix} \begin{pmatrix} c & c & c & \dots & c \end{pmatrix} = \begin{bmatrix} cx_1 & cx_1 & cx_1 & \dots & cx_1 \\ cx_2 & cx_2 & \dots & cx_2 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ cx_n & \dots & \dots & \dots & cx_n \end{bmatrix}$$

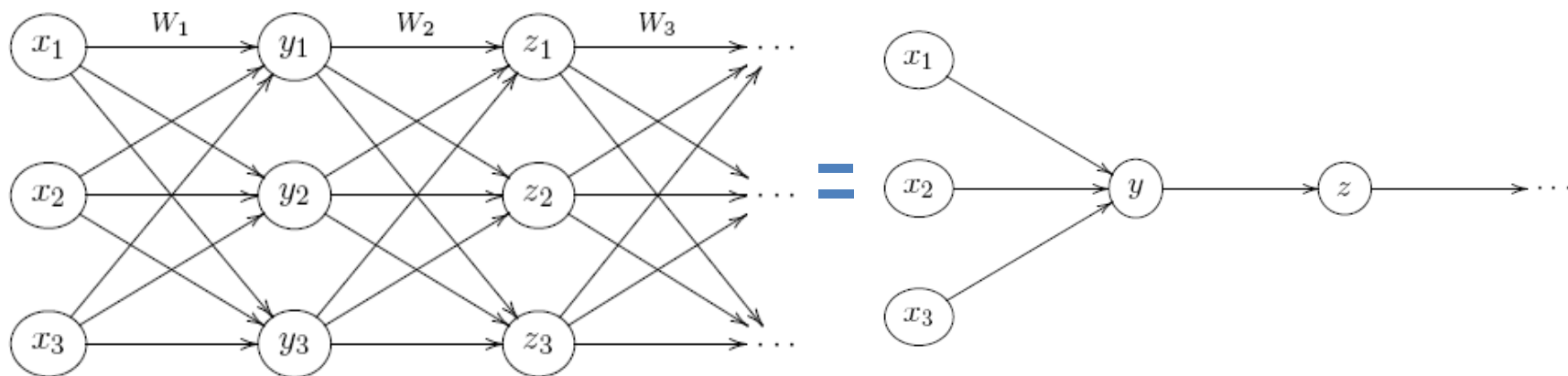
w 가 같은 값으로만 채워짐.

Vanishing Gradient (지속적인 감소)



WT 主叫: Expressive limitation

到 $\gamma = \frac{1}{2}$



$$W_1 = \begin{pmatrix} \omega_1 & \omega_1 & \omega_1 \\ \omega_2 & \omega_2 & \omega_2 \\ \omega_3 & \omega_3 & \omega_3 \end{pmatrix}, \quad W_2 = \begin{pmatrix} \omega' & \omega' & \omega' \\ \omega' & \omega' & \omega' \\ \omega' & \omega' & \omega' \end{pmatrix}, \quad W_3 = \begin{pmatrix} \omega'' & \omega'' & \omega'' \\ \omega'' & \omega'' & \omega'' \\ \omega'' & \omega'' & \omega'' \end{pmatrix}, \quad \dots$$

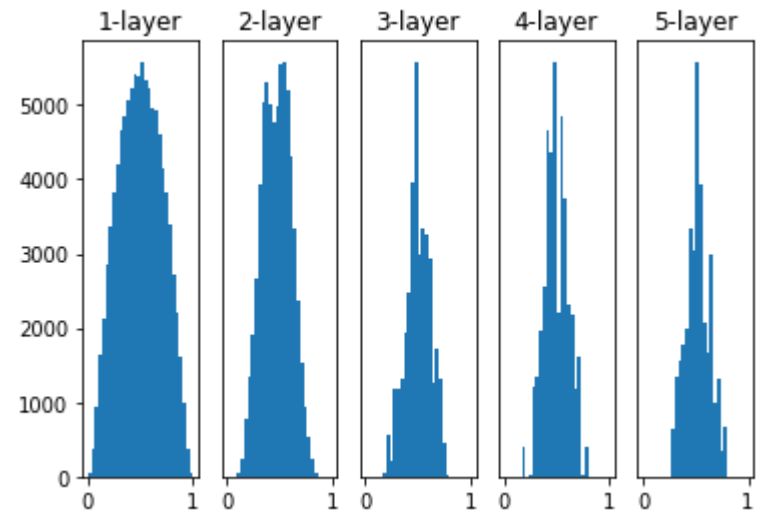
$$W_1 = \begin{pmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \end{pmatrix}, \quad W_2 = (3\omega'), \quad W_3 = (3\omega''), \quad \dots$$

LeCun, Xavier initialization

LeCun: $W \sim N\left(0, \frac{1}{n_{in}}\right)$

Xavier: $W \sim N\left(0, \frac{2}{n_{in} + n_{out}}\right)$

n_{in} : # of input, n_{out} : # of output



LeCun initialization

활성화 함수가 sigmoidal 인 경우 사용.

Le Carré. Xavier

$$XW + b = \sum w_i X_i + b.$$

$|w| = 0$ 이거나 너무 크면 안 좋음

$\therefore$ 이 계층에 X_i 가 많을수록

$|w|$ 가 커질수 밖에 없음.

또 X_i 수가 커질수록 w_i 값은 줄어든다.

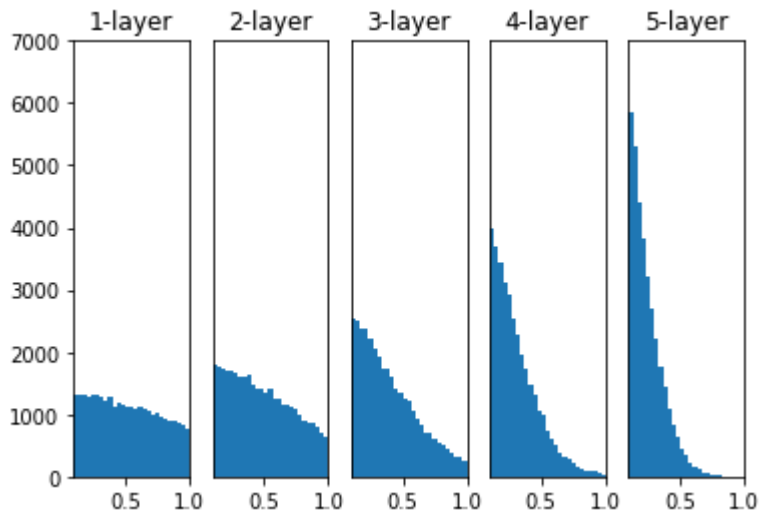
$$w_i \sim \mathcal{N}\left(0, \frac{1}{n \text{ 신호}}\right)$$

$$\frac{1}{n_i n}$$

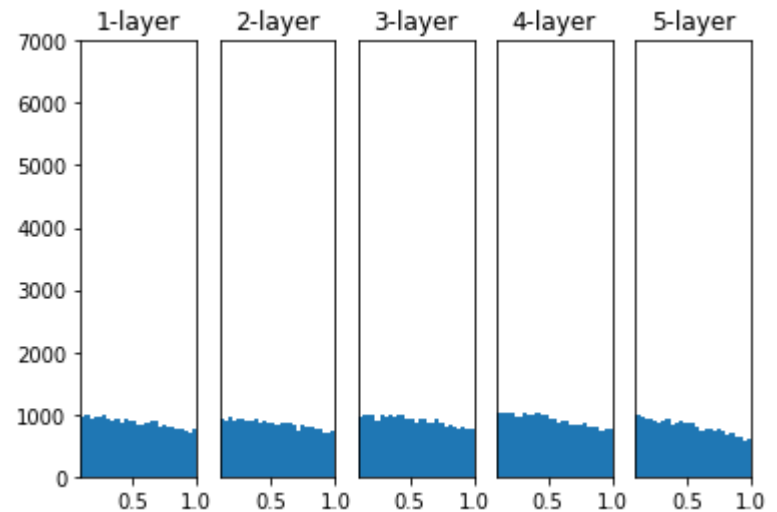
He initialization

$$\text{He: } W \sim N\left(0, \frac{2}{n_{in}}\right)$$

활성화 함수가 ReLU 일 때

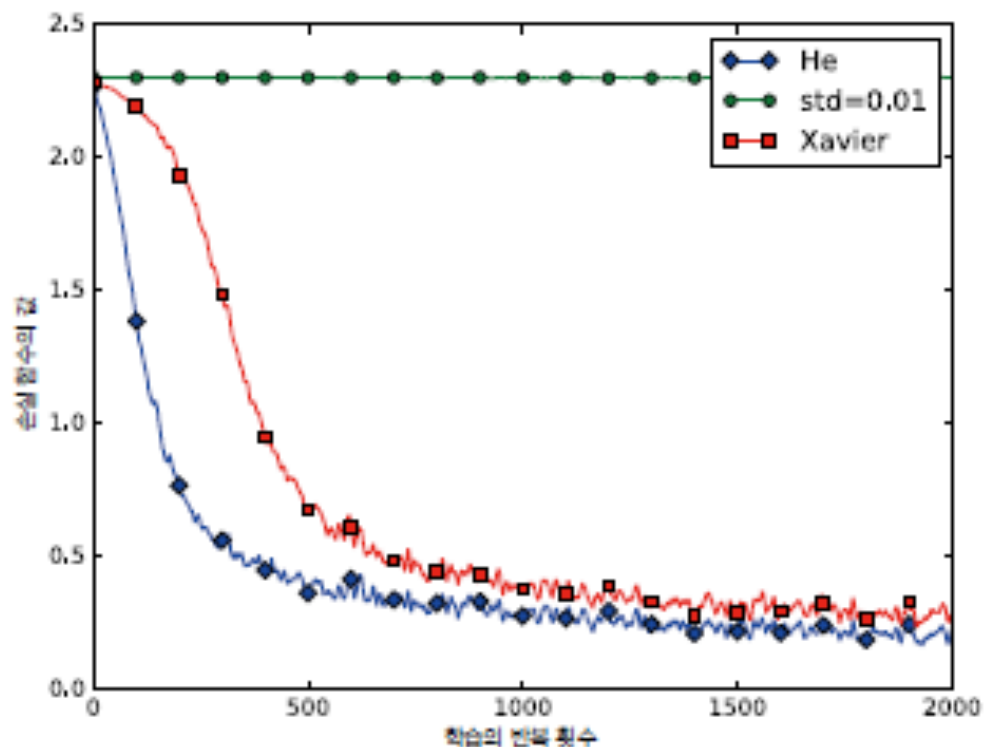


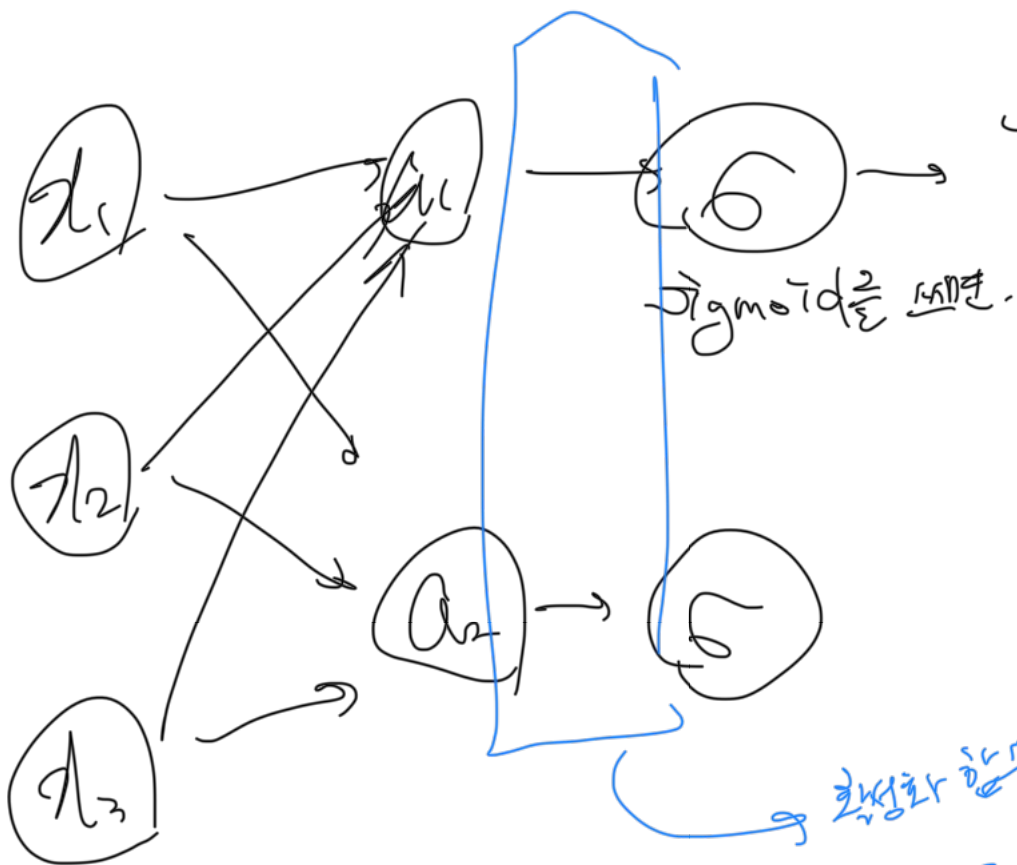
LeCun



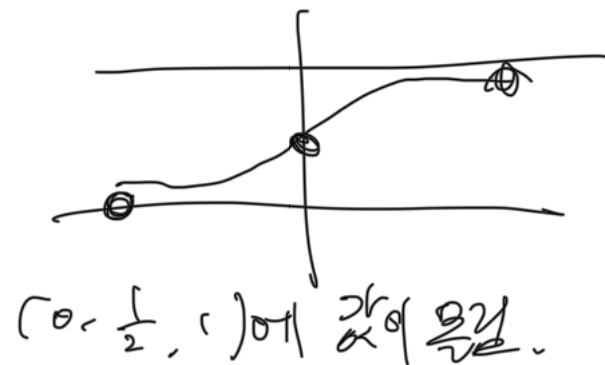
He

Initialization 비교





sigmoid를 쓰면.



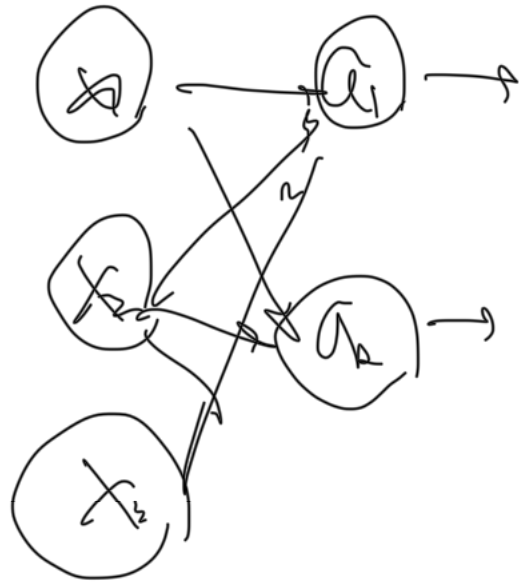
$(0, \frac{1}{2}, 1)$ 에 값이 문질.

→ 활성화 함수에 sigmoid에
데이터를 잘 변환하게 가공

∴ 정규화 (normalization)

$$\frac{a_1x_1 + a_2x_2 + a_3x_3 + b}{\text{affine layer}}$$

Example



$$X = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \\ 2 & 2 & 0 \\ 2 & 1 & 1 \end{bmatrix} \quad W = \begin{bmatrix} 1 & 2 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} \quad b = (0, 1)$$

$$X \cdot W + b = \begin{pmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \\ 4 & 8 \\ 5 & 10 \end{pmatrix} + \begin{pmatrix} 0 & 1 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 3 \\ 2 & 5 \\ 3 & 7 \\ 4 & 9 \\ 5 & 11 \end{pmatrix}$$

결과를 얻었다.

$\begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{pmatrix}$
 $\begin{pmatrix} 3 \\ 5 \\ 7 \\ 9 \\ 11 \end{pmatrix}$

μ_1
 μ_2

$$\therefore (\mu_1, \mu_2) = (3, 7)$$

정리

i) ~~오차~~ ~~평균~~ ~~변동~~ ~~분산~~ ~~표준~~ ~~편차~~ (3.1)

ii) 오차를 구한다: $X - \mu$

iii) (오차)²을 구한다: $(X - \mu)^2$

iv) 분산을 구한다 (오차)²의 평균: (2.8)

v) 표준편차를 구한다 $\sigma = (12, 18)$

$$X - \mu = \begin{pmatrix} 1 & 3 \\ 2 & 5 \\ 3 & 7 \\ 4 & 9 \\ 5 & 11 \end{pmatrix} - \begin{pmatrix} 1 & 7 \\ 2 & 9 \\ 3 & 7 \\ 4 & 9 \\ 5 & 7 \end{pmatrix} = \begin{pmatrix} -2 & -4 \\ -1 & -2 \\ 0 & 0 \\ 1 & 2 \\ 2 & 4 \end{pmatrix}$$

$$(X - \mu)^2 = \begin{pmatrix} 4 & 16 \\ 1 & 4 \\ 0 & 0 \\ 1 & 4 \\ 4 & 16 \end{pmatrix} \quad \sigma^2 = (2.8)$$

$\mu_1 = 3$
 $\mu_2 = 7$

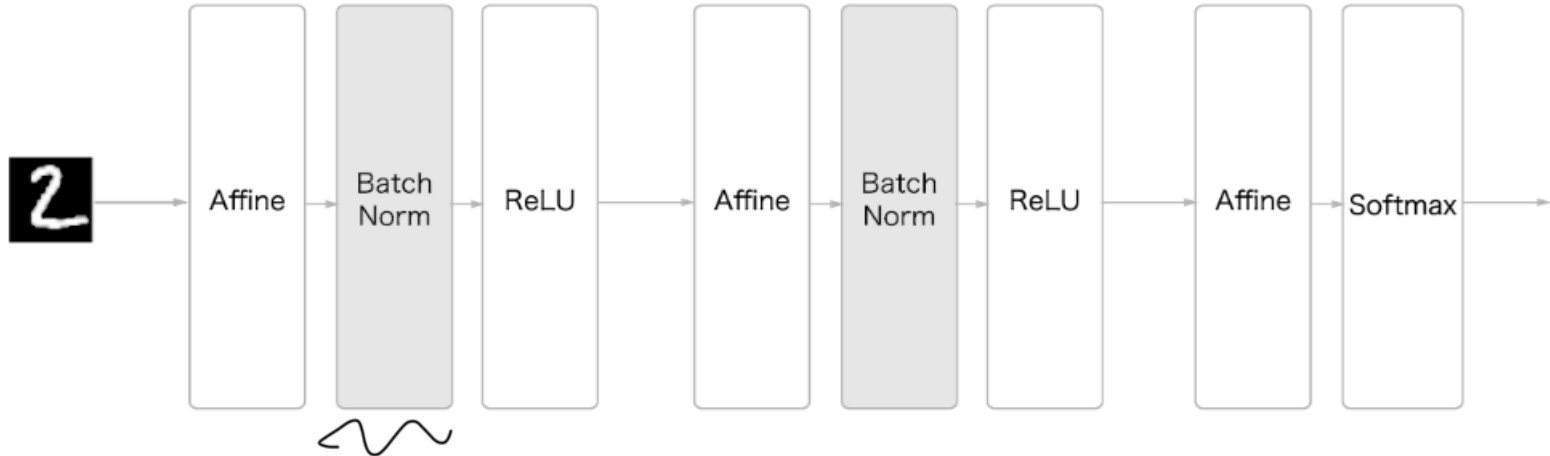
$$X - \mu = \frac{X - \mu}{\sigma}$$

정규 분포 Parameter
 분산 bias

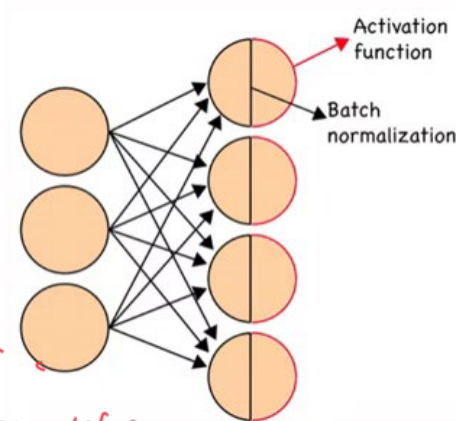
2차원

$$X \sim N(\mu, \sigma^2)$$

Batch Normalization



$$\underbrace{\gamma X_n + \beta}_{(\gamma, \beta)}$$



Affine Layer를 추가하지 않고

적응값의 (γ, β) 파라미터를 더 높은 성능을 낸다.

Affine 보다는 적응값

Batch Normalization

feature 개수 D 인 데이터 N 개

$$\mathbf{x} = \begin{pmatrix} x_1^{(1)} & x_2^{(1)} & \cdots & x_D^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \cdots & x_D^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{(N)} & x_2^{(N)} & \cdots & x_D^{(N)} \end{pmatrix} : \text{배치 묶음, } N \times D \text{ 행렬}$$

$$\boldsymbol{\mu} = (\mu_1 \quad \mu_2 \quad \cdots \quad \mu_D) : \text{열에 대한 평균, 길이 } D \text{인 벡터}$$

$$\mathbf{x}_c = \mathbf{x} - \boldsymbol{\mu} : \text{각 열의 평균을 0으로 만듦, } N \times D \text{ 행렬}$$

$$\boldsymbol{\sigma}^2 = (\sigma_1^2 \quad \sigma_2^2 \quad \cdots \quad \sigma_D^2) : \text{각 열의 분산, 길이 } D \text{인 벡터}$$

$$\mathbf{x}_N = \frac{\mathbf{x}_c}{\boldsymbol{\sigma}} : \text{각 열을 normalize함, } N \times D \text{ 행렬}$$

$$\boldsymbol{\gamma} = (\gamma_1 \quad \gamma_2 \quad \cdots \quad \gamma_D) : \text{길이 } D \text{인 벡터, 각 행에 곱}$$

$$\boldsymbol{\beta} = (\beta_1 \quad \beta_2 \quad \cdots \quad \beta_D) : \text{길이 } D \text{인 벡터, 각 행에 합}$$

$$\sim \mathcal{N}(\boldsymbol{\beta}, \boldsymbol{\sigma}^2)$$

Backpropagation (+ batch normalization)

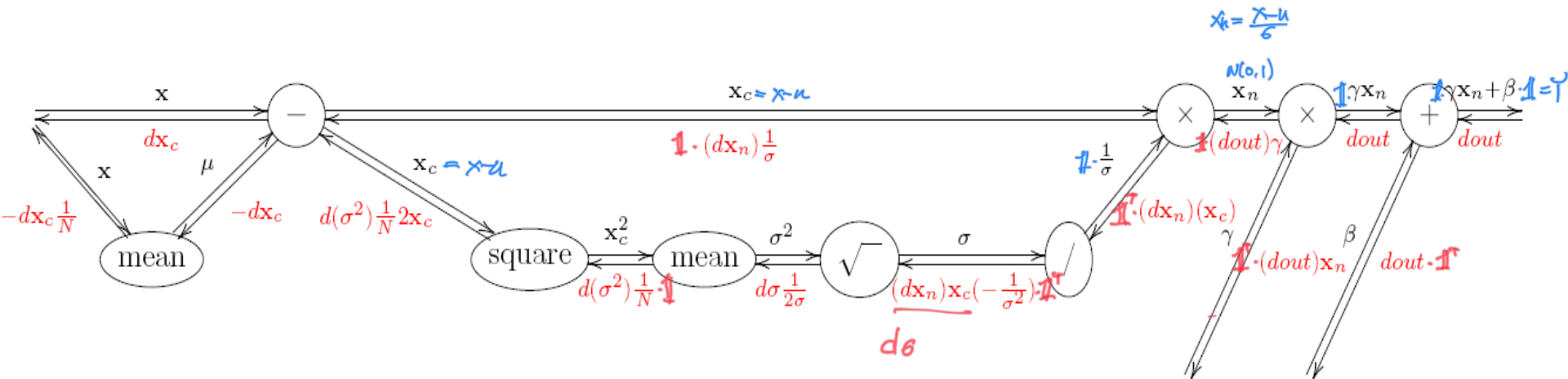


Diagram 1: A node containing a division symbol $\div$. An input x enters from the left, and an output $\frac{1}{x} = y$ exits to the right.

$$\frac{\partial L}{\partial y} \propto \frac{\partial y}{\partial x}$$

$$= \frac{\partial}{\partial y} (-y^2)$$

Diagram 2: A node containing a multiplication symbol $*$ and the number 2. An input x enters from the left, and an output $x^2 = y$ exits to the right.

$$\frac{\partial L}{\partial y} = \frac{\partial y}{\partial x}$$

$$= \frac{\partial}{\partial y} (2x)$$

Diagram 3: A node containing a square root symbol $\sqrt{}$. An input x enters from the left, and an output $\sqrt{x} = y$ exits to the right.

$$\frac{\partial L}{\partial y} \propto \frac{\partial y}{\partial x}$$

$$= \frac{\partial}{\partial y} \left(\frac{1}{2y} \right)$$

Diagram 4: A node labeled "mean" with a circle containing $\frac{1}{N}$. An input x enters from the left, and an output $\frac{1}{N} \cdot \mathbf{1}^T x = y$ exits to the right.

$$\frac{\partial L}{\partial y} = \left(\frac{1}{N} \cdot \mathbf{1} \right)$$

$$\therefore \frac{\partial L}{\partial x} = \frac{1}{N} \cdot \mathbf{1} \cdot \frac{\partial L}{\partial y}$$

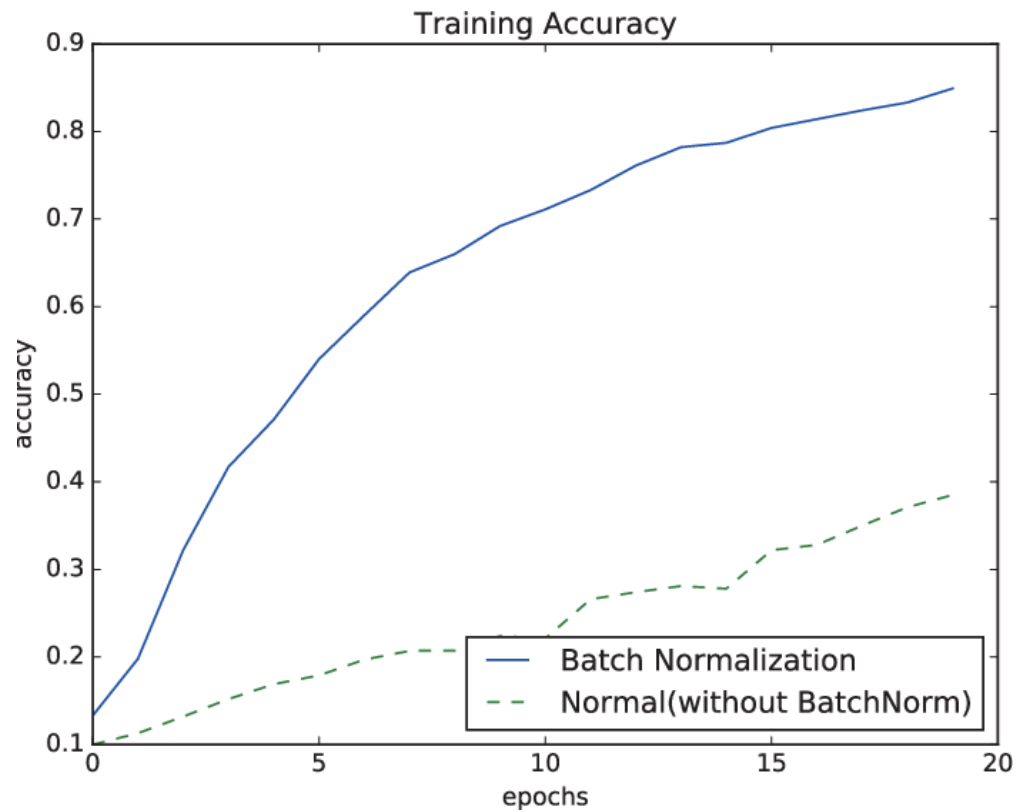
Diagram 5: A node labeled "repeat" with a circular arrow. An input x enters from the left, and an output $\mathbf{1} \cdot x = y$ exits to the right.

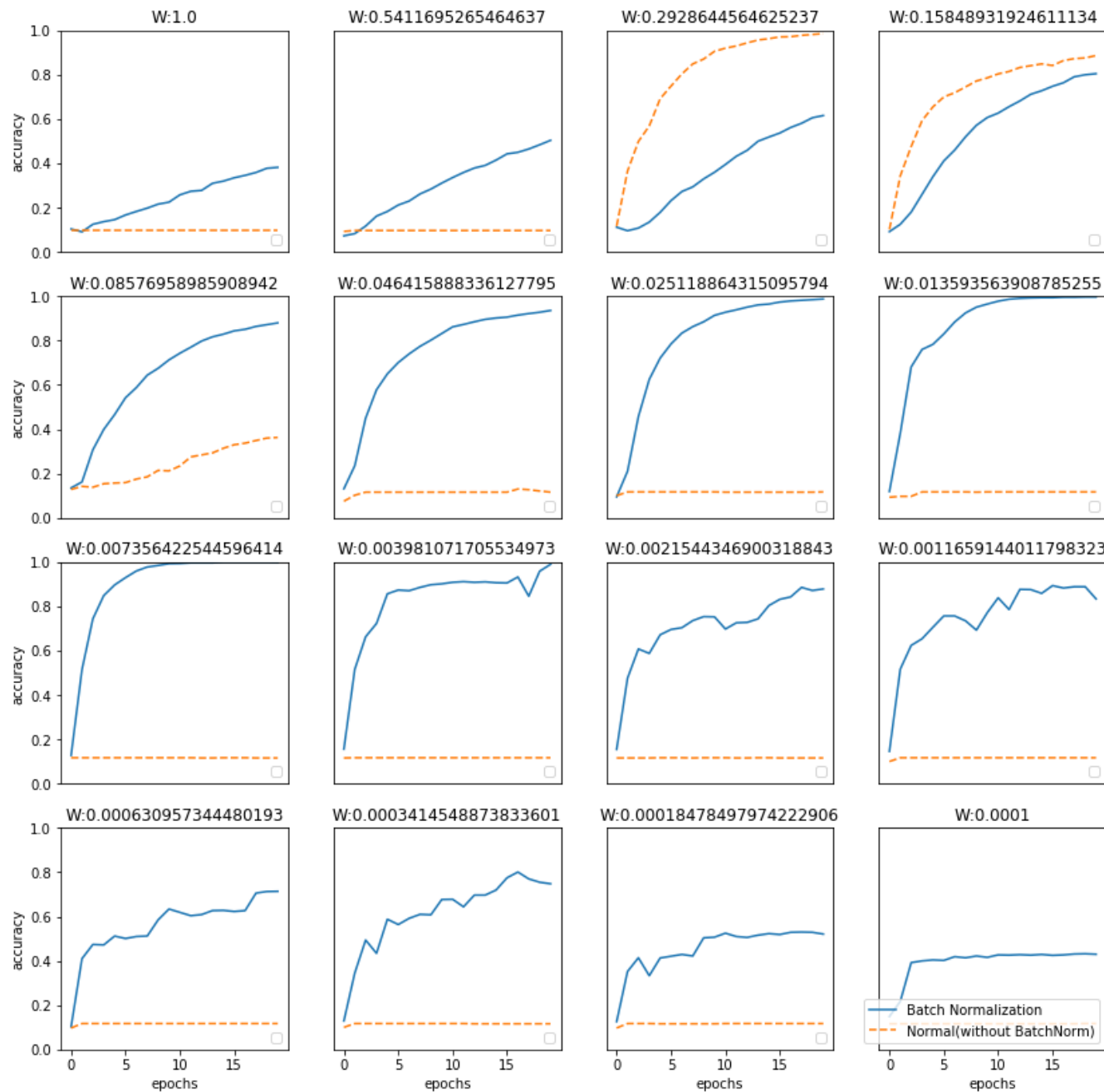
$$\frac{\partial L}{\partial y} = (\mathbf{1}^T)$$

$$\therefore \frac{\partial L}{\partial x} = \mathbf{1}^T \cdot \frac{\partial L}{\partial y}$$

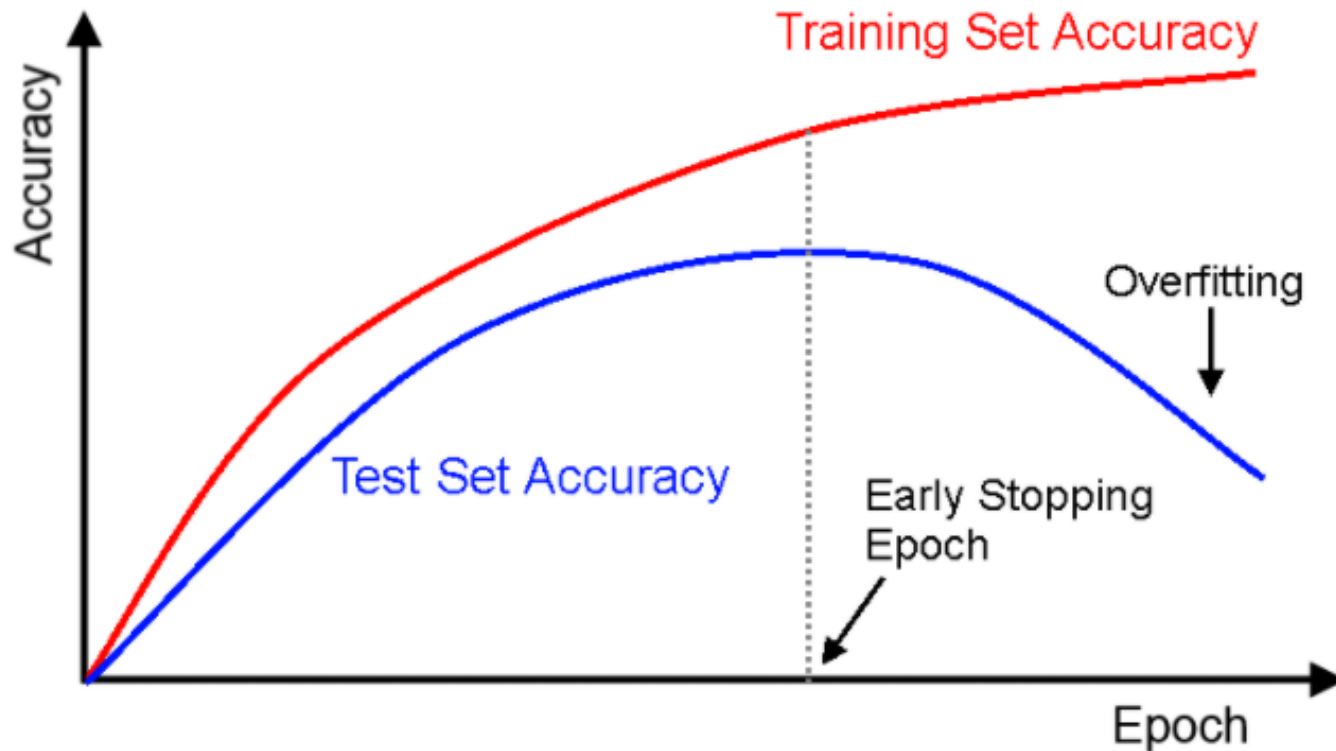
$$\mathbf{1} = \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}$$

학습속도 개선
초기값에 덜 의존
overfitting 억제





Overfitting

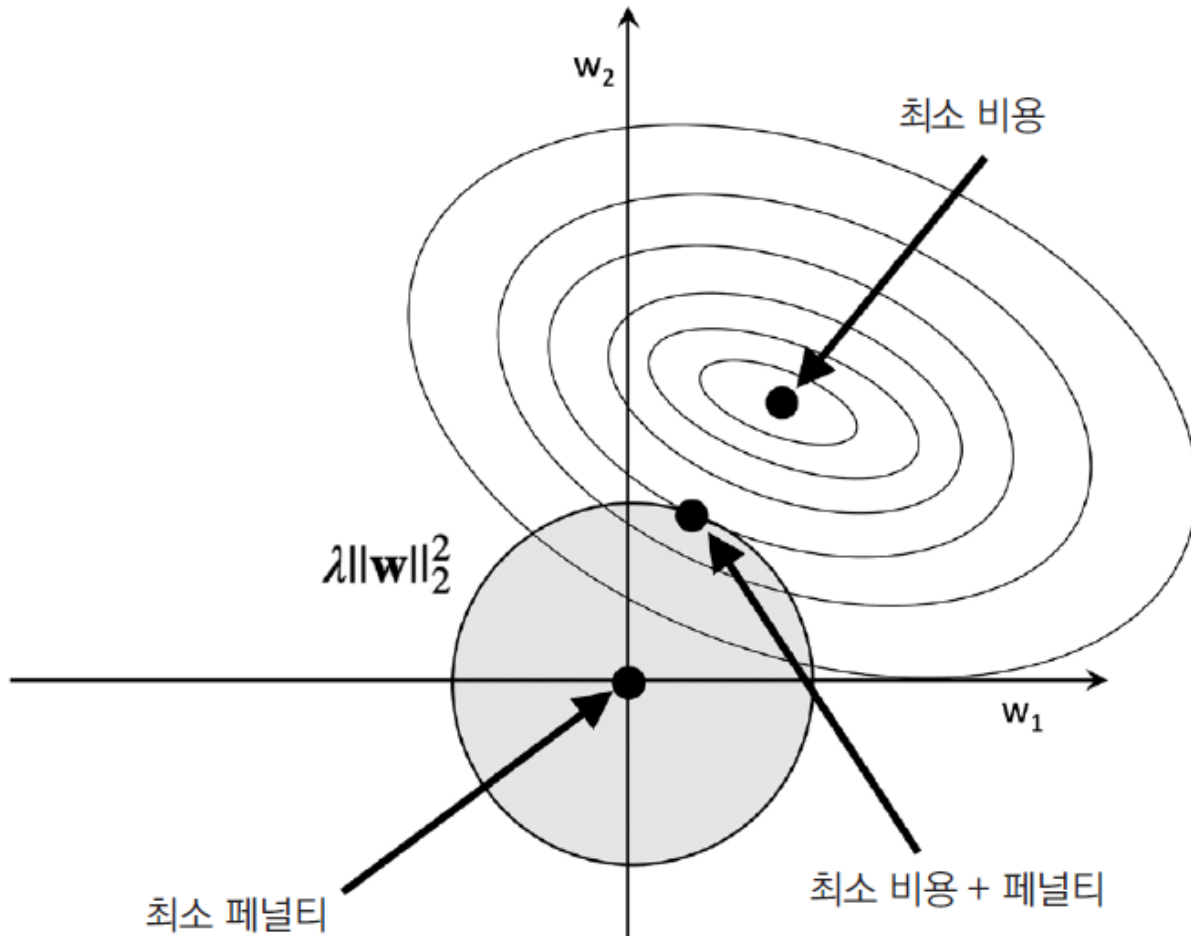


Regularizer (Weight decay)

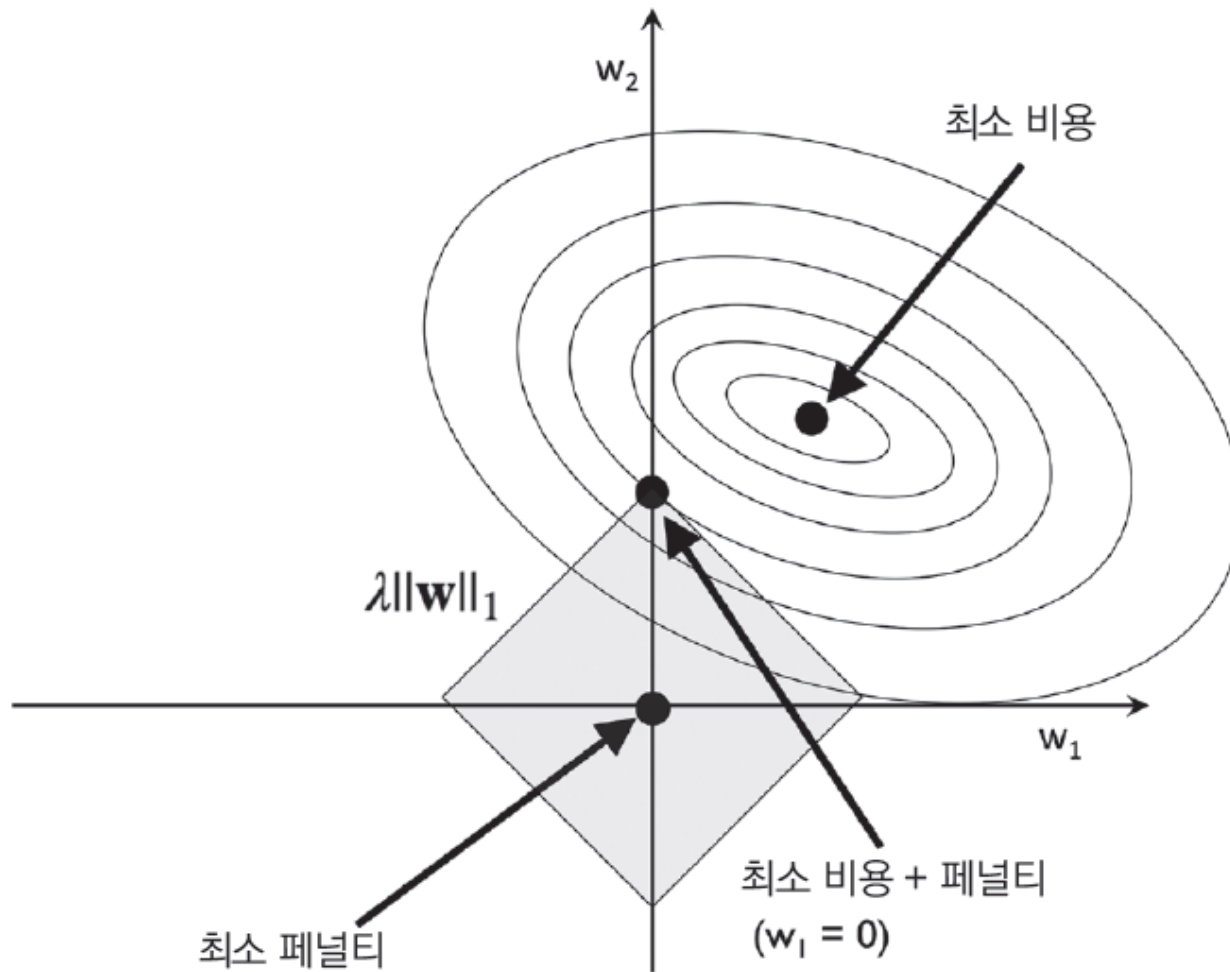
$$L2: \|\mathbf{w}\|_2^2 = \sum_{j=1}^m w_j^2$$

$$L1: \|\mathbf{w}\|_1 = \sum_{j=1}^m |w_j|$$

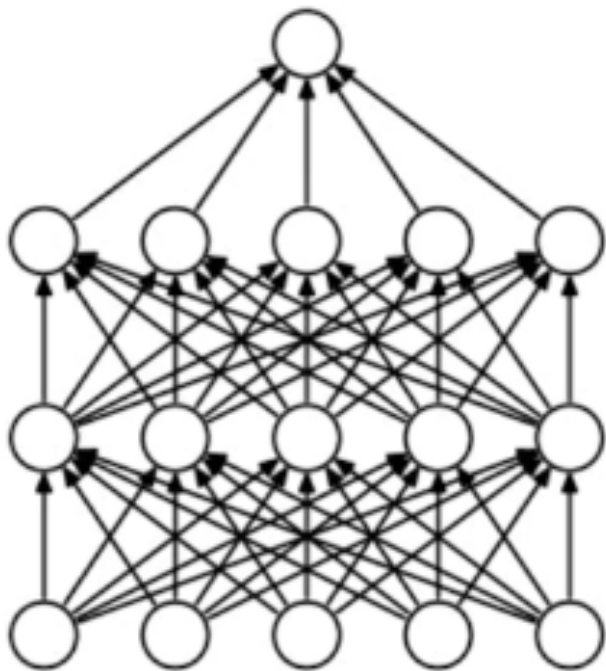
Regularizer (L2 norm)



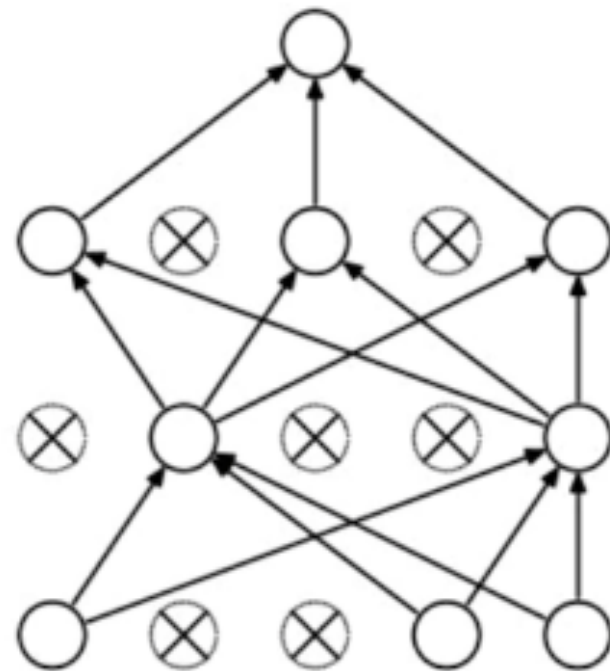
Regularizer (L1 norm)



Dropout

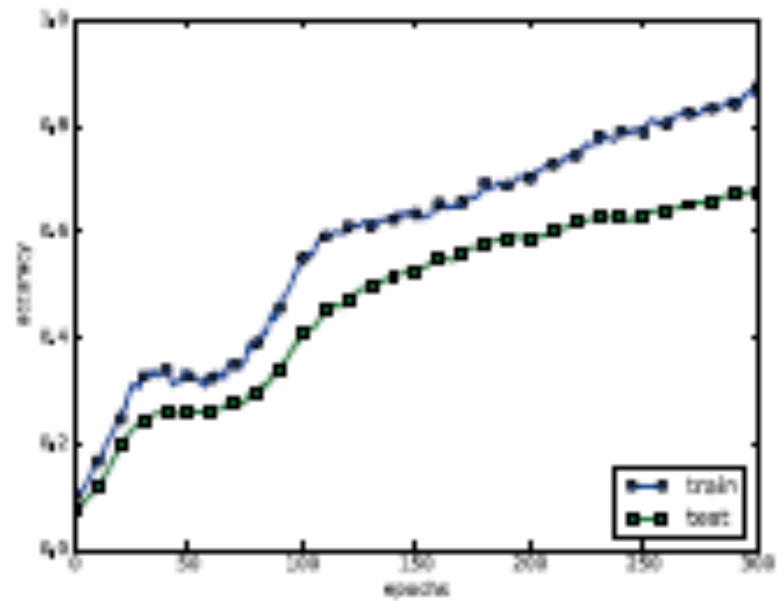
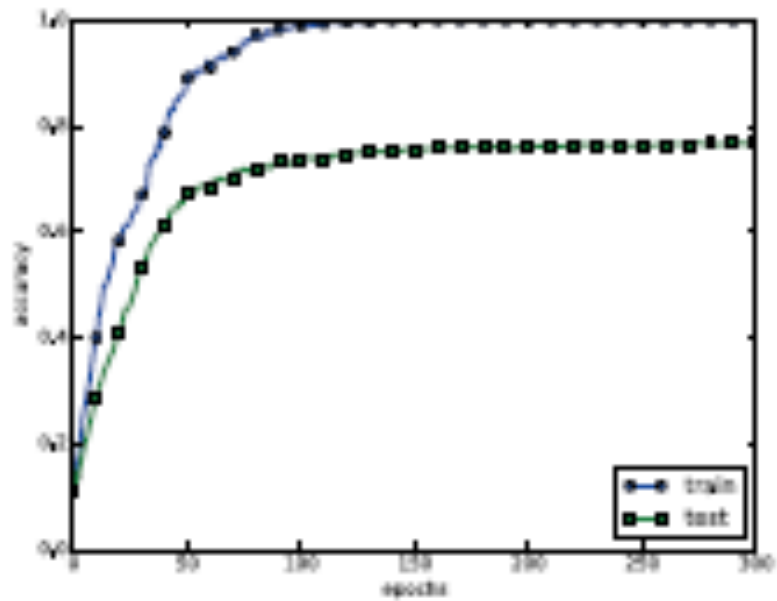


(a) Standard Neural Net



(b) After applying dropout.

Dropout



Hyperparameter

Parameter - weight

Hyperparameter

ex) 층의 개수, 뉴런의 개수, learning rate, 손실함수의 종류, 배치 크기, 훈련 회수, optimizer의 종류와 관련된 계수값, 가중치 초기값, 가중치 감소 계수, dropout 비율

- 좋은 값을 찾기 위해서는 여러 값을 직접 시도해보는 수 밖에 없다.
- 일부 hyperparameter는 특정값이 추천되거나 실제 사용되는 값이 고정

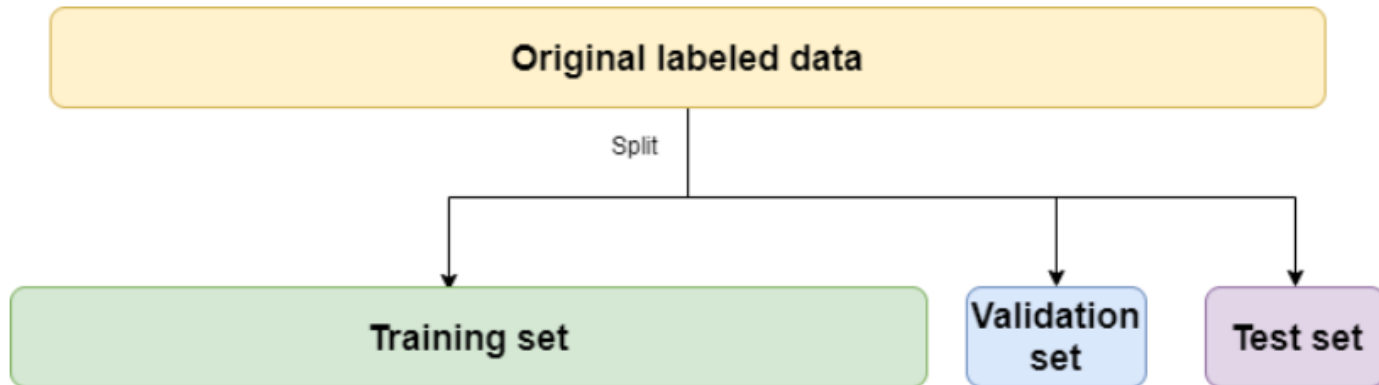
ex) Momentum계수 0.9, Adam은 $\beta_1 = 0.9, \beta_2 = 0.999$

Validation data

train 데이터 : parameter 학습

validation 데이터 : hyperparameter 성능 평가

test 데이터 : 신경망의 범용 성능 평가



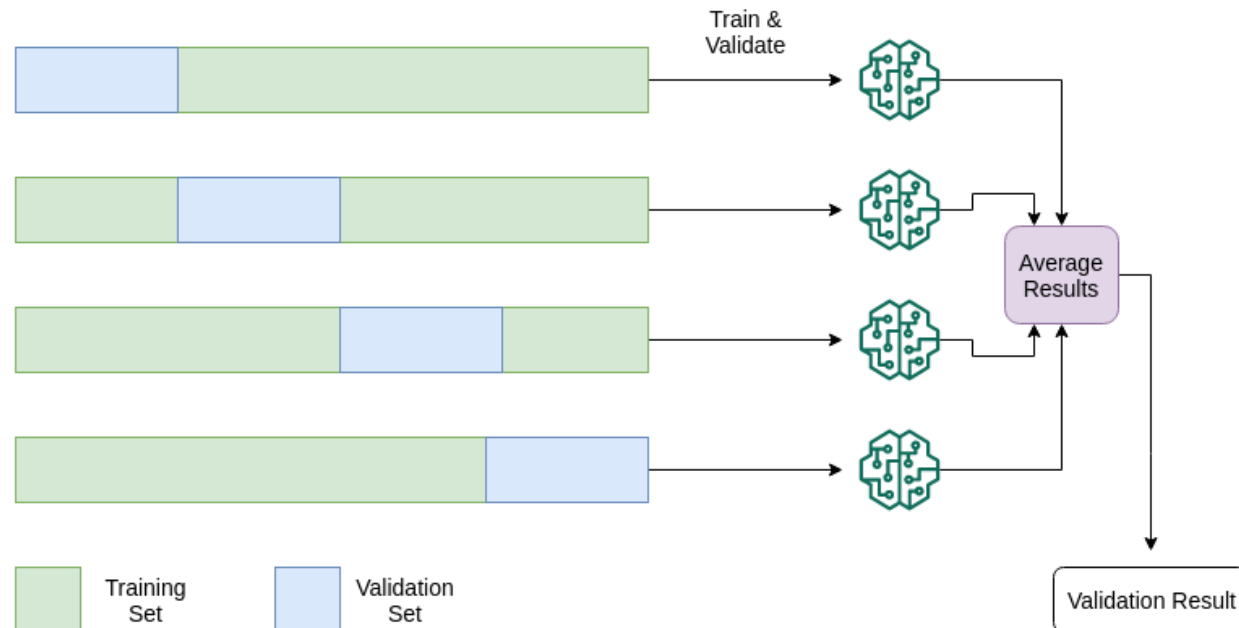
K-fold validation

train 데이터 K-등분

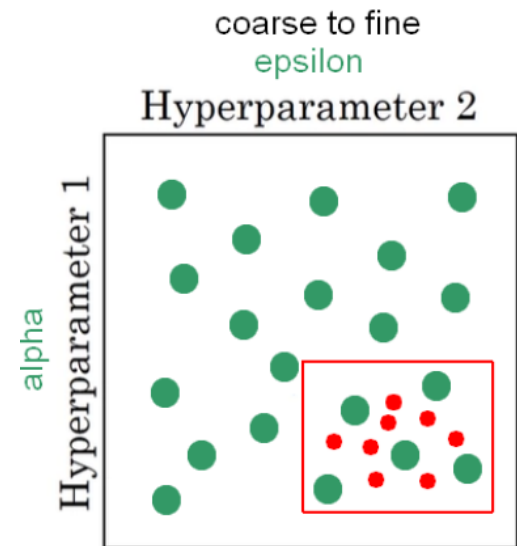
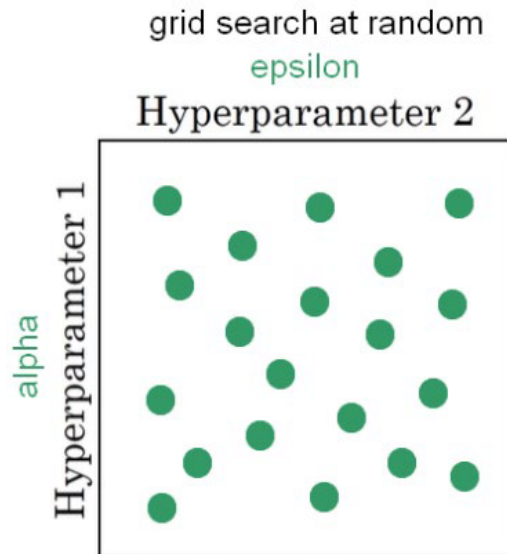
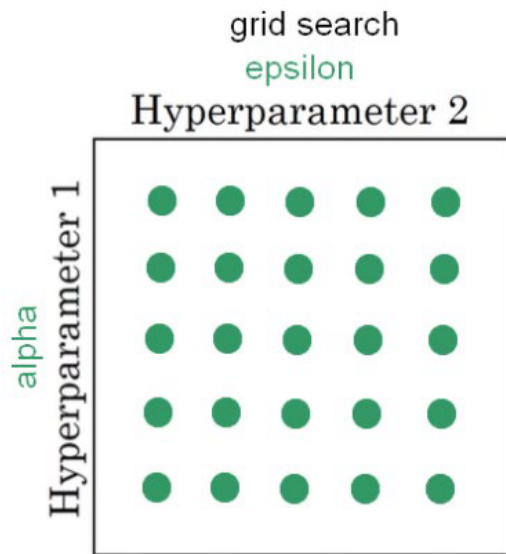
하나를 검증용으로 나머지를 훈련용으로 K 번 반복

평균

4-fold validation



Hyperparameter



Question?

자료 출처

Deep learning from scratch, 한빛미디어, 사이토고키

https://github.com/youbeebee/deeplearning_from_scratch